

Phase 6: User Interface Development

AgriConnect

Salesforce-Based Farmer Support & Marketplace System

Step 1: Lightning App Builder

➤ Theory:

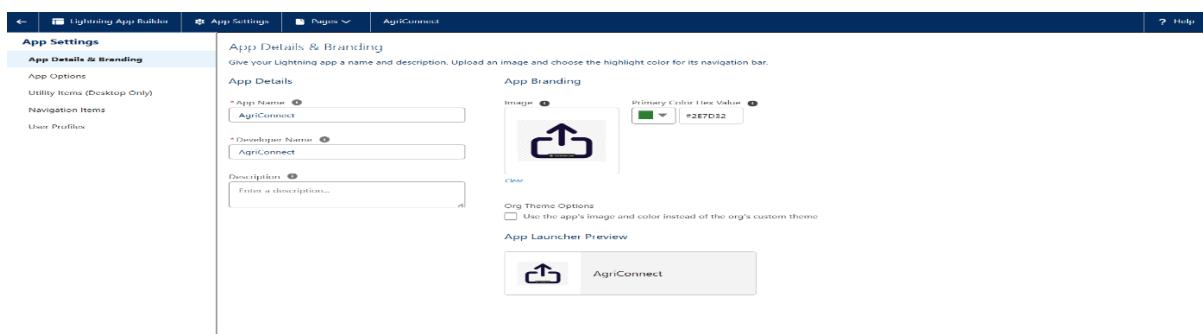
- Lightning App Builder allows you to create **custom pages** in Salesforce.
- You can design **App Pages, Record Pages, Home Pages**, and include **standard or custom components** (like LWC).
- Pages can be activated for **org, app, or profile-specific use**.

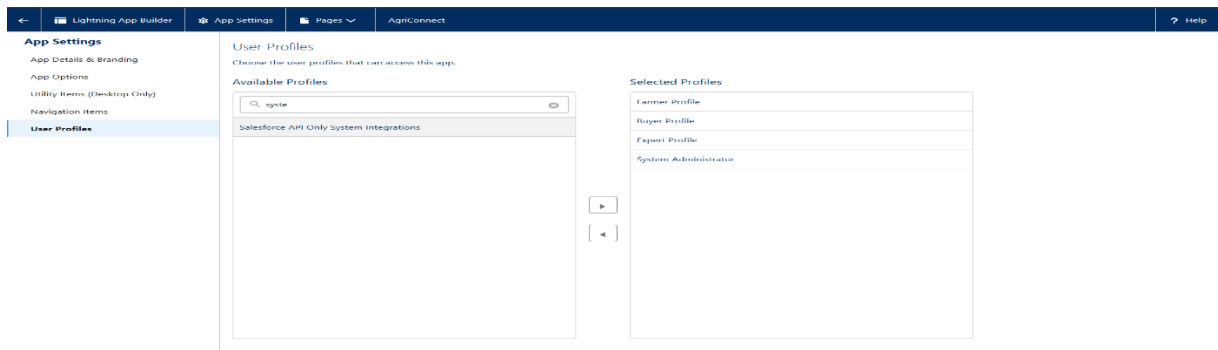
➤ Steps to implement:

- Setup → Lightning App Builder → New → Choose Page Type (App, Record, or Home).
- Drag components to regions (Header, Main, Sidebar).
- Save and **Activate** the page.

➤ Image placement:

- Screenshot of **Lightning App Builder dashboard**
- Example of **dragging components on a page**





Step 2: Record Pages

➤ Theory:

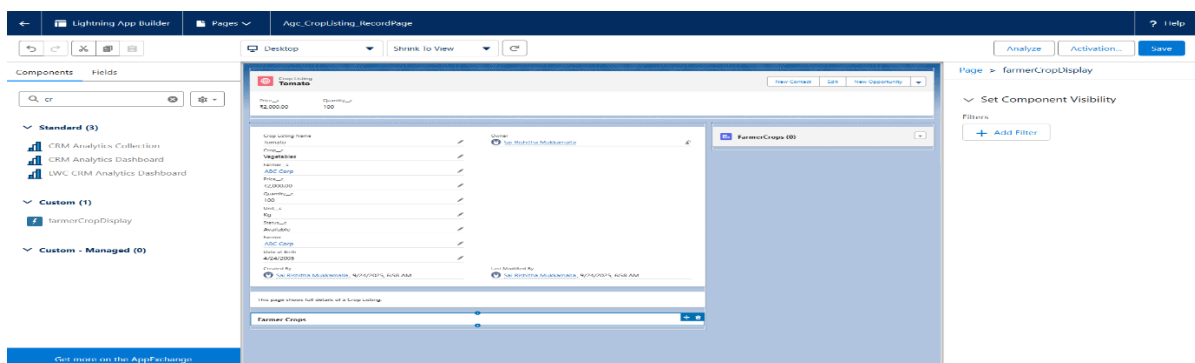
- Record Pages define the layout of **individual object records**.
- Can include **key fields, Rich Text fields, LWCs, and Related Lists**.
- Supports **dynamic actions and conditional visibility**.

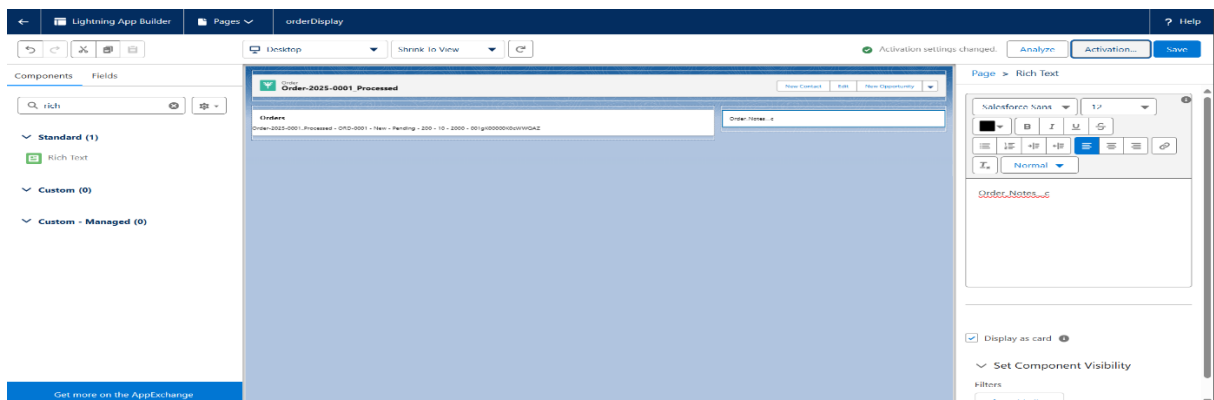
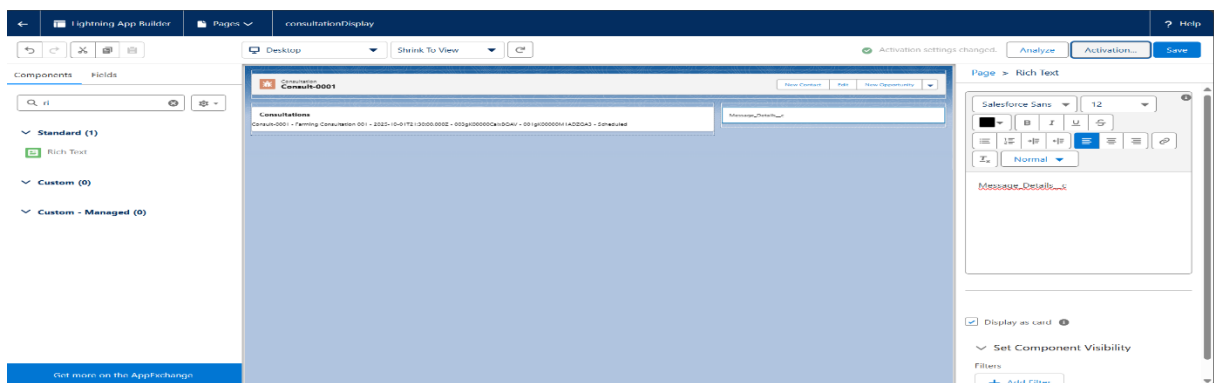
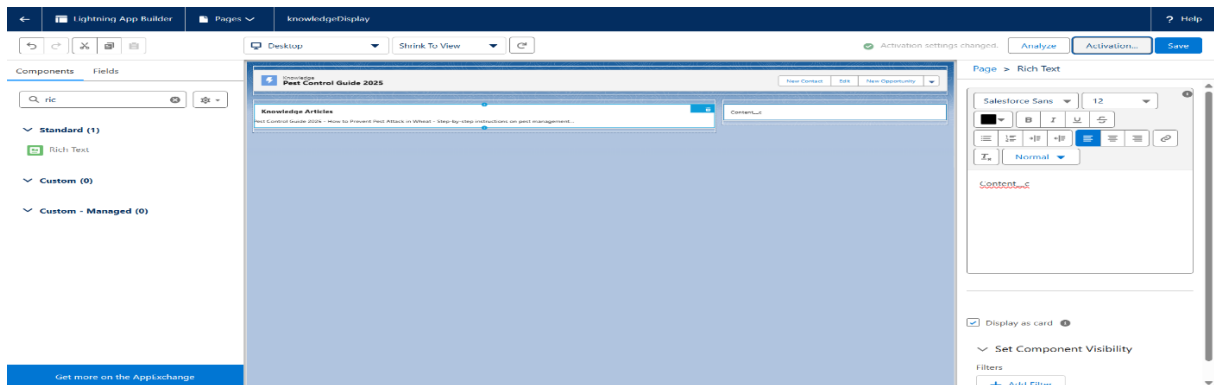
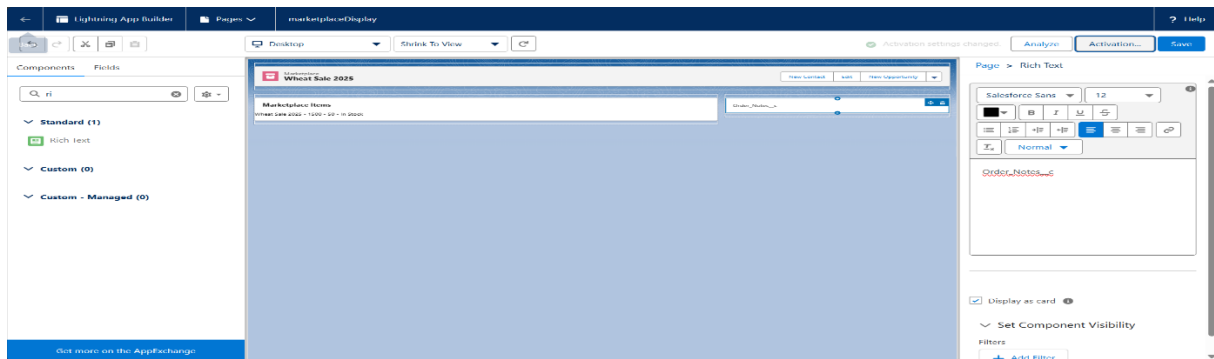
➤ Steps:

- Setup → Object Manager → [Object] → Lightning Record Pages → New / Edit.
- Drag components: Header → Main → Sidebar.
- Bind **LWCs or standard fields** to each region.
- Save and Activate.

➤ Image placement:

- Screenshot of **Record Page layout with LWC**
- Highlight **Header, Main, Sidebar regions**





Step 3: Tabs

➤ Theory:

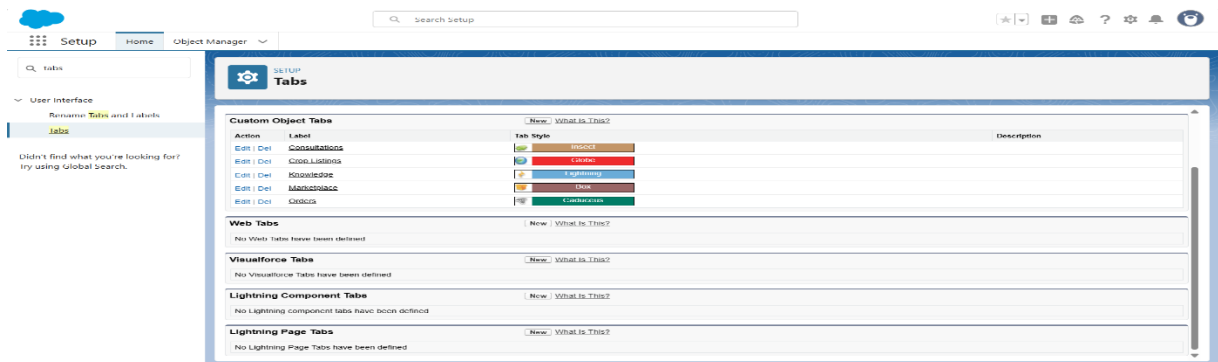
- Tabs provide **navigation in apps** for standard objects or **custom LWCs**.
- Types: Object Tabs, Lightning Component Tabs (for LWCs).

➤ **Steps:**

- Setup → Tabs → New → Choose type (Object / Lightning Component).
- Fill **Label, Name, Icon**.
- Add to App Navigation via App Manager.

➤ **Image placement:**

- Screenshot of **Tabs creation page**
- App navigation bar showing **added tabs**



Step 4: Home Page Layouts

➤ **Theory:**

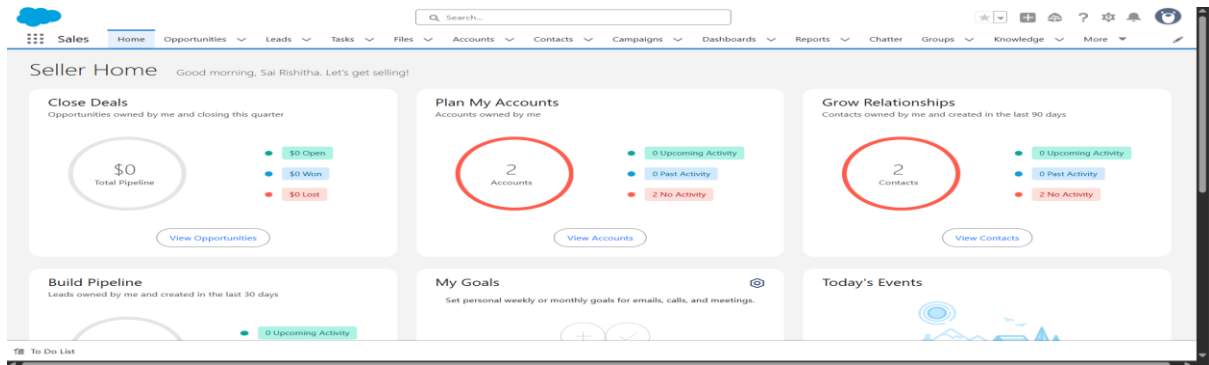
- Home Pages are **default pages when users open the app**.
- Can include **dashboard components, recent records, or custom LWCs**.

➤ **Steps:**

- Setup → Lightning App Builder → Home Page → New.
- Drag components (List Views, Charts, LWCs).
- Save and Activate → Assign to Profiles / App.

➤ **Image placement:**

- Screenshot of **Home Page layout**
- Example of **recent records / charts in main region**



Step 5: Utility Bar

➤ **Theory:**

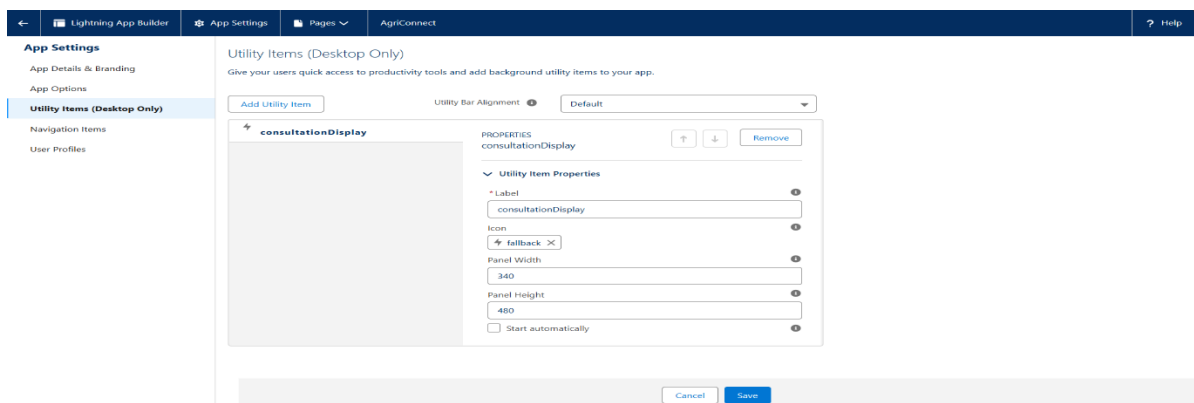
- Utility Bar allows **global access to tools** like search, notifications, and quick actions.

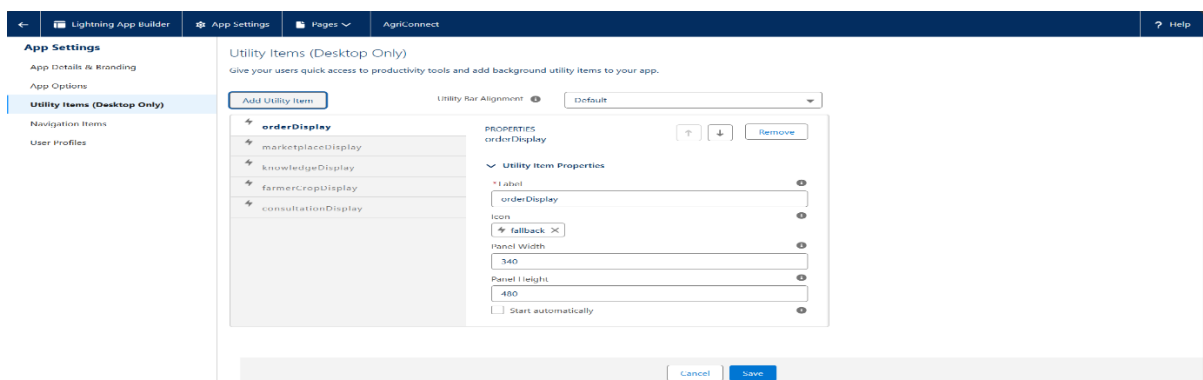
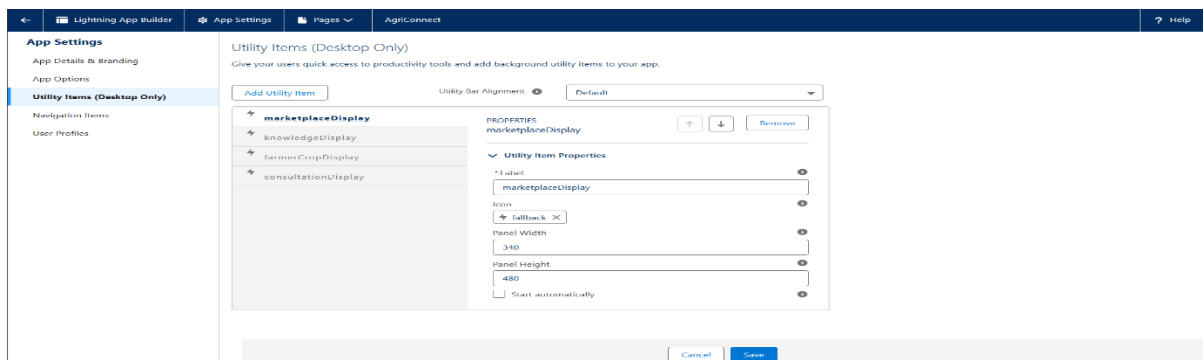
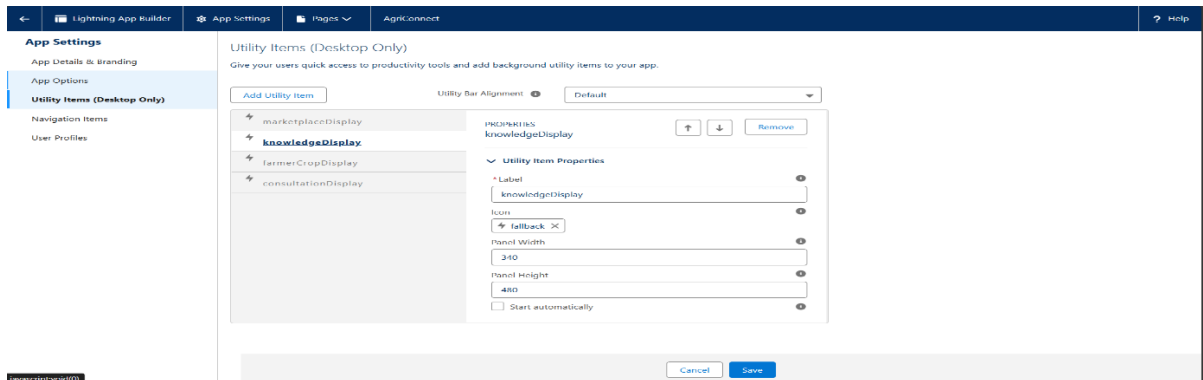
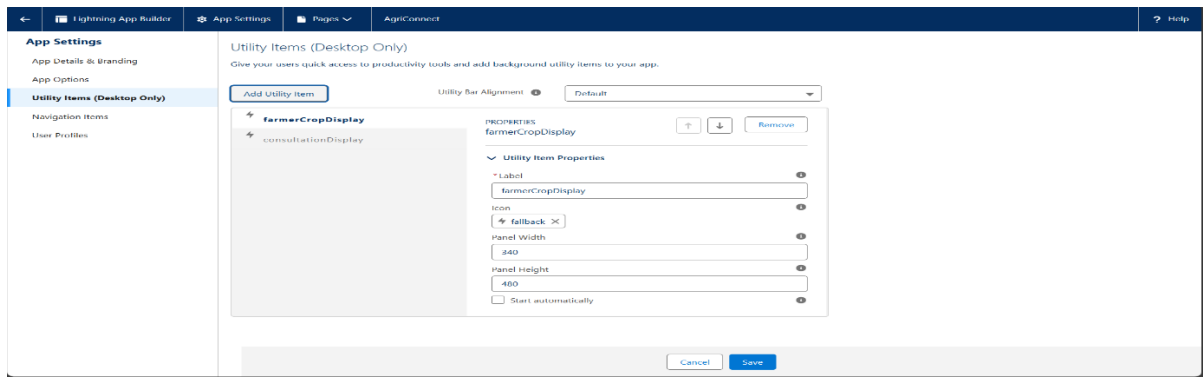
➤ **Steps:**

- Setup → App Manager → Edit App → Utility Bar → Add components.
- Save and Activate.

➤ **Image placement:**

- Screenshot of **Utility Bar with components**





Step 6: LWC (Lightning Web Components)

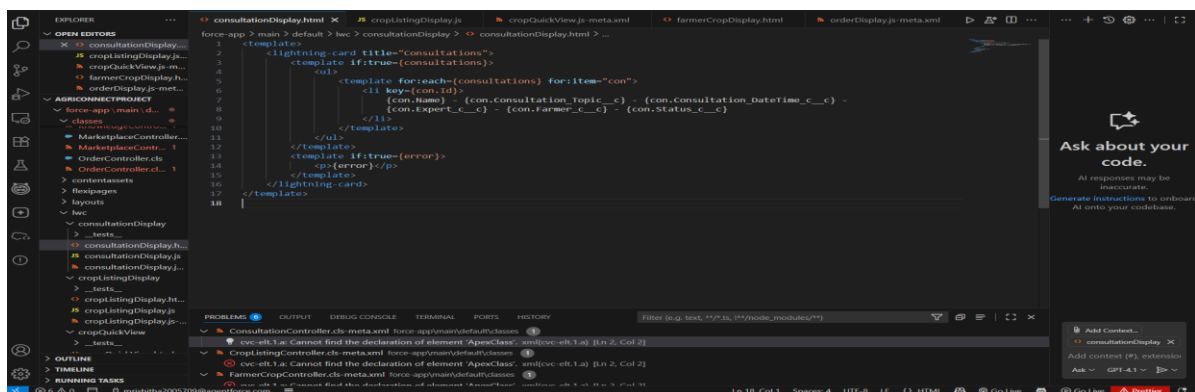
➤ Theory:

- LWC is modern framework for building UI in Salesforce.

- Can be used in Record Pages, App Pages, Home Pages, or tabs.
- Components consist of .html, .js, .js-meta.xml.

➤ Image placement:

- Screenshot of VS Code project structure showing LWC folders
- LWC component in Lightning App Builder



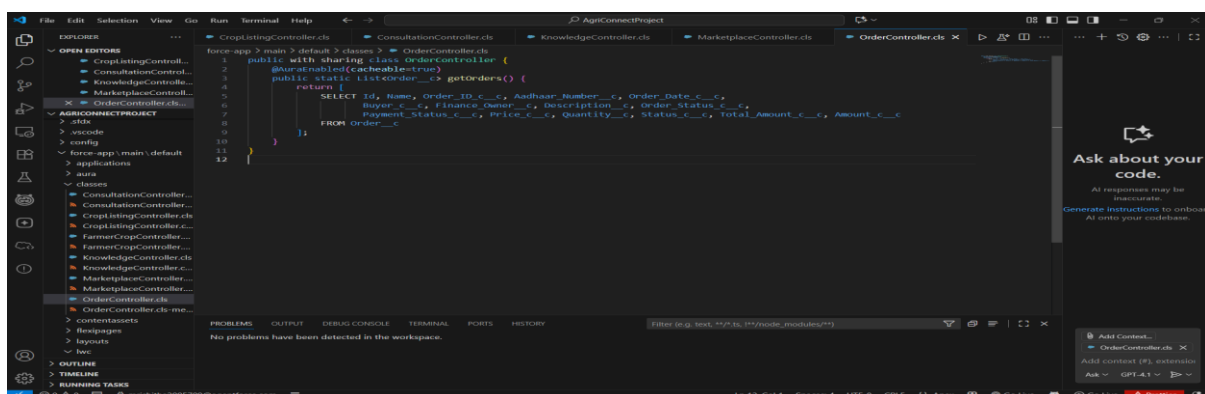
Step 7: Apex with LWC

➤ Theory:

- Apex methods can be called from LWC to fetch or update Salesforce data.
- Use `@AuraEnabled(cacheable=true)` for read-only and `@AuraEnabled` for DML operations.

➤ Image placement:

- Screenshot of Apex class in VS Code
- LWC calling Apex and displaying data



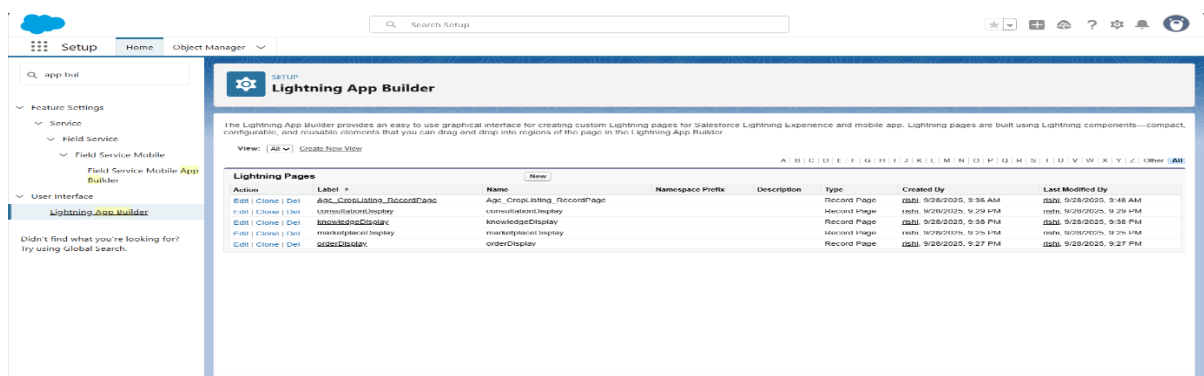
Step 8: Events in LWC

➤ Theory:

- LWCs communicate using Custom Events:
 - Child → Parent (CustomEvent)
 - Application-level events for global communication

➤ Image placement:

- Screenshot of LWC JS code showing dispatchEvent



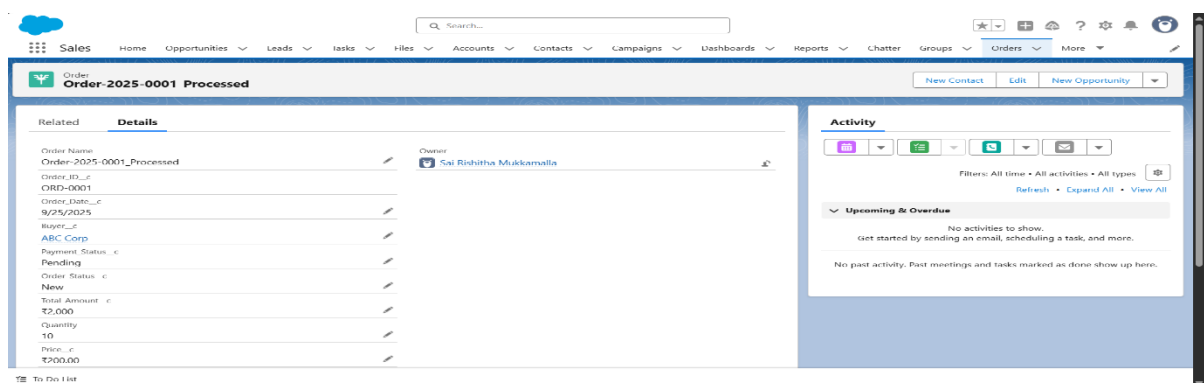
Step 9: Wire Adapters

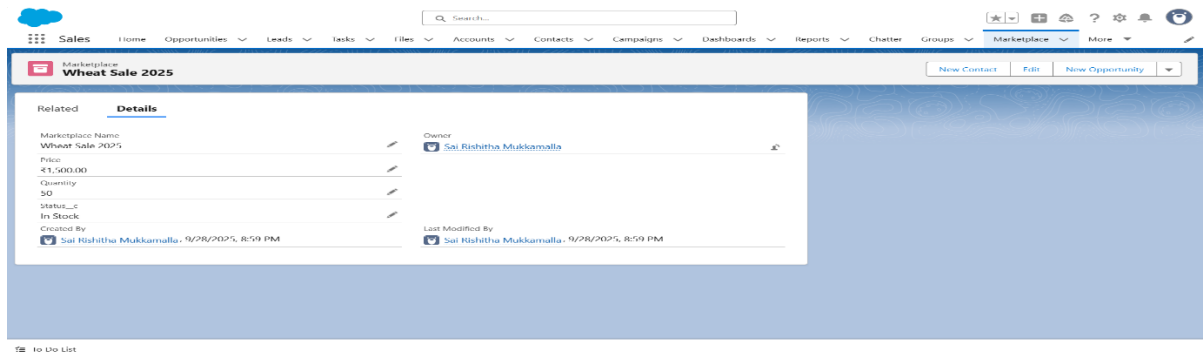
➤ Theory:

- @wire decorator reactively fetches data from Salesforce objects or Apex methods.
- Automatically updates the component when the data changes.

➤ Image placement:

- Example screenshot of @wire code in VS Code





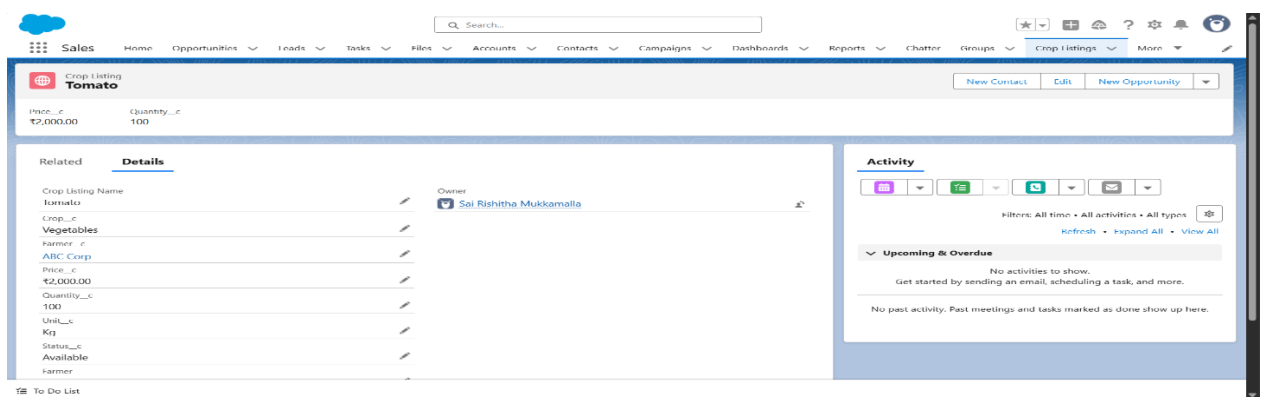
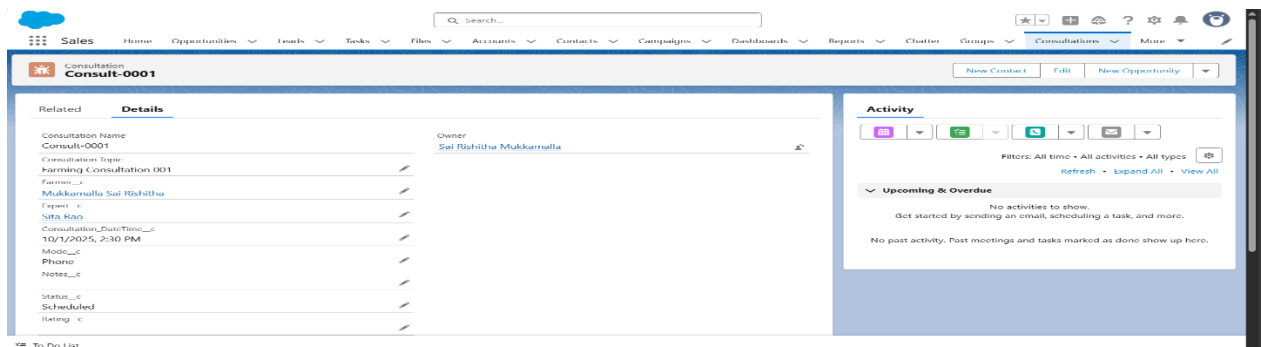
Step 10: Imperative Apex Calls

➤ Theory:

- Used when data fetch must occur on-demand, not automatically.
- Usually triggered by button click or user action in LWC.

➤ Image placement:

- Screenshot of button triggering Apex call in LWC

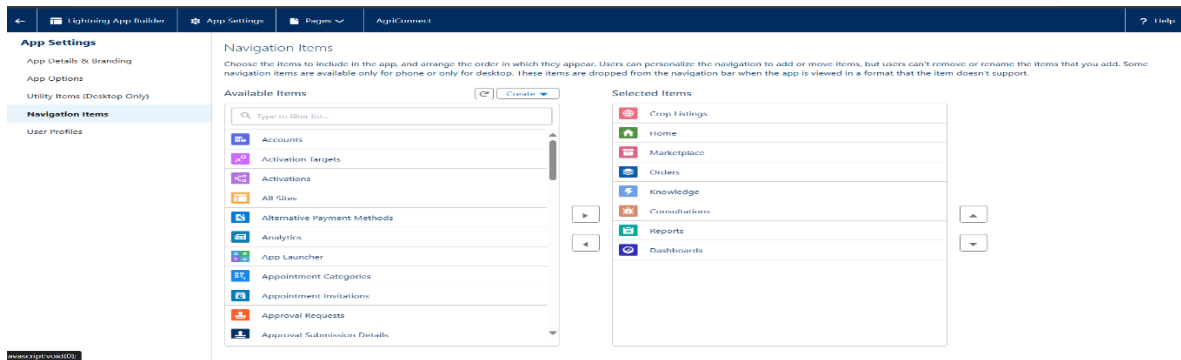


Step 11: Navigation Service

➤ Theory:

- NavigationMixin allows programmatic navigation in LWCs.

- Can navigate to Record Pages, List Views, App Pages, or URLs.
- Image placement:
- Screenshot of navigation code and resulting page navigation



Conclusion

- Phase 6: User Interface Development focused on transforming the backend logic and data model into an interactive and user-friendly experience. Using Lightning App Builder, we designed custom record pages, home pages, and utility bars that improve navigation and usability. With the addition of tabs, users can quickly access key objects like Marketplace, Orders, Knowledge, and Consultation.
- Through Lightning Web Components (LWC) integrated with Apex, we created dynamic and responsive interfaces that fetch, display, and update data in real time. Features such as events in LWC, wire adapters, and imperative Apex calls allow seamless communication between components and the Salesforce platform. The Navigation Service further enhances our ability by enabling smooth movement between records, lists, and apps.
- Overall, Phase 6 ensured that the system is not only functional but also intuitive and efficient, providing a modern UI aligned with Salesforce standards. This phase bridges the gap between data and end-users, ensuring that farmers, experts, and administrators can access and manage information easily.